Donald Van Steenwyk

ISTM 622 M4 P2P Clustering Milestone Deliverable

1. Video Link: Clickable URL to your Unlisted YouTube video. https://youtu.be/6q3lAR-Ldjg

2. Honor Code & AI Attestation.
On my honor, as an Aggie, I have neither given nor received unauthorized assistance on this assignment. I further affirm that I have not and will not provide this code to any person, platform, or repository, without the express written permission of Dr. Gomillion. I understand that any violation of these standards will have serious repercussions. The only AI assistance I used was the Google search summary and Claude. I followed the links provided in the summary to verify it was not hallucination and tested any code it helped me write to ensure it worked correctly. Claude helped me understand the reflection question better so that I could put it in my own words.

3. Configuration Scripts: Provide your 60-galera.cnf (or similar) configuration block to prove you understand the setup.

You can find the 60-galera.cnf config script by searching for '/root/galera_config.sh'
You can ignore the AWS CLI section of the script and focus on the contents of STARTUP to see the User Data script. If you want to run this in the AWS CLI, make sure to change SGNAME to the name of your security group for this assignment or a security group will be created for you. You can also change the USERNAME and PASSWORD if you would like too. Then just copy and paste it into the lab terminal.

```
#!/bin/bash
# TODO make things consistent and pretty
# (This will probably never happen)

ip_permissions_str() { #$1=SGID $2=PROTOCOL $3=PORT $4=MYUSERID
    echo "IpProtocol=$2,FromPort=$3,ToPort=$3, \
        UserIdGroupPairs=[{UserId=$4,GroupId=$1}]" \
        | tr -d "[:space:]"
}

read -r -d '' STARTUP <<'EOF'
#!/bin/bash
# shebang so the shell knows this is a shell script
# use tail -f /var/log/user-data.log | grep "mariadb" to check mariadb status
# use tail -f /var/log/user-data.log | grep -m 1 "SUCCESS" if you are confident

set -e # Exit script if error
set -u # Unset variable references cause error -> exit
set -o # Catch pesky |bugs| hiding in pipes

# CHANGE THESE AFTER THE PRIMARY STARTS
SERVERID="1"
HOSTADDR="localhost"

# Log all stdout output to this log file
exec > >(tee /var/log/user-data.log|logger -t user-data -s 2>/dev/console) 2>&1
```

```
# Politely ask tools not to attempt to run interactively
export DEBIAN_FRONTEND=noninteractive

touch /root/1-script-started

# Standard update package lists and then packages
apt-get update
apt-get upgrade -y

touch /root/2-packages-upgraded

# Use curl to grab files from MariaDB website
apt-get install apt-transport-https curl -y

# Install MariaDB 11.8
# Configure the official gpg key from Maria DB Foundation
mkdir -p /etc/apt/keyrings
curl -o /etc/apt/keyrings/mariadb-keyring.pgp \
'https://mariadb.org/mariadb_release_signing_key.pgp'
# Add MariaDB Repository and Key Repository
cat > /etc/apt/sources.list.d/mariadb.sources <<SUBEOF
X-Repolib-Name: MariaDB
Types: deb
URIs: https://mirrors.accretive-networks.net/mariadb/repo/11.8/ubuntu
Suites: noble
Components: main main/debug
Signed-By: /etc/apt/keyrings/mariadb-keyring.pgp
SUBEOF

# The following broke for some reason I'm guessing an update
# Configure the official gpg key from Maria DB Foundation
#curl -LsSo /etc/apt/trusted.gpg.d/mariadb-keyring-2025.gpg \
#    https://supplychain.mariadb.com/mariadb-keyring-2025.gpg
# Use Maria DB Foundation's official setup script that varifies gpg key
# and adds the repo
#curl -LsS https://downloads.mariadb.com/MariaDB/mariadb_repo_setup \
#    > mariadb_repo_setup
# Official checksum verification and exit script on fail
#checksum="73f4ab14ccc3ceb8c03bb283dd131a3235cfc28086475f43e9291d2060d48c97"
#echo "${checksum} mariadb_repo_setup" | sha256sum -c -
#cat mariadb_repo_setup | bash

# update package list from mariadb repo
apt-get update
# install the mariadb-server and galera packages from the newly added repo
apt-get install -y mariadb-server galera-4

# MariaDB automatically enables and starts as a
# part of the Ubuntu package install process
systemctl stop mariadb
systemctl disable mariadb
```

```
touch /root/3-mariadb-installed

USERNAME="dvansteenwyk"
PASSWORD="notthepassword"

useradd -m -p ${PASSWORD} -s /bin/bash ${USERNAME}

touch /root/4-user-setup

apt-get install unzip -y
sudo -u ${USERNAME} cat > /home/${USERNAME}/etl.sql <<SUBEOF
DROP DATABASE IF EXISTS POS;
CREATE DATABASE POS;
USE POS;

CREATE TABLE Customer (
    id SERIAL PRIMARY KEY,
    firstName VARCHAR(32),
    lastName VARCHAR(30),
    email VARCHAR(128),
    address1 VARCHAR(100),
    address2 VARCHAR(50),
    phone VARCHAR(32),
    birthdate DATE,
    zip DECIMAL(5) ZEROFILL
) ENGINE=InnoDB; /* Don't forget to explicitly set engine */

CREATE TABLE City (
    zip DECIMAL(5) ZEROFILL PRIMARY KEY,
    city VARCHAR(32),
    \`state\` VARCHAR(4)
) ENGINE=InnoDB;

CREATE TABLE \`Order\` (
    id SERIAL PRIMARY KEY,
    datePlaced DATE,
    dateShipped DATE,
    customer_id BIGINT UNSIGNED
) ENGINE=InnoDB;

CREATE TABLE Orderline (
    order_id BIGINT UNSIGNED NOT NULL,
    product_id BIGINT UNSIGNED NOT NULL,
    quantity INTEGER,
    PRIMARY KEY (order_id, product_id)
) ENGINE=InnoDB;

CREATE TABLE Product (
    id SERIAL PRIMARY KEY,
    name VARCHAR(128),
    currentPrice DECIMAL(6,2),
    availableQuantity INTEGER
) ENGINE=InnoDB;
```

```
CREATE TABLE PriceHistory (
    id SERIAL PRIMARY KEY,
    oldPrice DECIMAL(6,2),
    newPrice DECIMAL(6,2),
    ts TIMESTAMP,
    product_id BIGINT UNSIGNED
) ENGINE=InnoDB;


/* Avoid defining FKs in CREATE because this is less painful */
ALTER TABLE Customer ADD CONSTRAINT FK_Customer_City_zip
    FOREIGN KEY (\`zip\`)
    REFERENCES \`City\`(\`zip\`);
ALTER TABLE \`Order\` ADD CONSTRAINT FK_Order_Customer_customer_id
    FOREIGN KEY (\`customer_id\`)
    REFERENCES \`Customer\`(\`id\`);
ALTER TABLE \`Orderline\` ADD CONSTRAINT FK_Orderline_Order_order_id
    FOREIGN KEY (\`order_id\`)
    REFERENCES \`Order\`(\`id\`);
ALTER TABLE \`Orderline\` ADD CONSTRAINT FK_Orderline_Product_product_id
    FOREIGN KEY (\`product_id\`)
    REFERENCES \`Product\`(\`id\`);
ALTER TABLE \`PriceHistory\` ADD CONSTRAINT FK_PriceHistory_Product_product_id
    FOREIGN KEY (\`product_id\`)
    REFERENCES \`Product\`(\`id\`);

/* Create a table for the raw data with types suited for them */
CREATE TABLE CustomerStaging (
    id BIGINT UNSIGNED PRIMARY KEY,
    firstName VARCHAR(32),
    lastName VARCHAR(30),
    city VARCHAR(32),
    \`state\` VARCHAR(4),
    zip VARCHAR(10),
    address1 VARCHAR(100),
    address2 VARCHAR(50),
    email VARCHAR(128),
    birthdate VARCHAR(20)
);

LOAD DATA LOCAL INFILE '/home/${USERNAME}/customers.csv'
    INTO TABLE CustomerStaging FIELDS TERMINATED BY ','
        OPTIONALLY ENCLOSED BY '"'
    IGNORE 1 LINES;
/* Use a select to transform the data as it is more readable
   than load data substitutions */
INSERT INTO City (
    SELECT CAST(zip AS DECIMAL(5)),
            TRIM('"' FROM city),
            \`state\`
        FROM CustomerStaging
        GROUP BY zip
```

```
);
INSERT INTO Customer (
    SELECT id,
            TRIM('"' FROM firstName),
            TRIM('"' FROM lastName),
            NULLIF(email, ''),
            TRIM('"' FROM address1),
            NULLIF(TRIM('"' FROM address2), ''),
            NULL,
            STR_TO_DATE(birthdate, '%m/%d/%Y'),
            CAST(zip AS DECIMAL(5))
        FROM CustomerStaging
);

DROP TABLE CustomerStaging; /* Done with it */

/* Create a table for the raw data with types suited for them */
CREATE TABLE OrderStaging (
    id BIGINT UNSIGNED,
    customer_id BIGINT UNSIGNED,
    datePlaced VARCHAR(30),
    dateShipped VARCHAR(30)
);
LOAD DATA LOCAL INFILE '/home/${USERNAME}/orders.csv'
    INTO TABLE OrderStaging FIELDS TERMINATED BY ','
        OPTIONALLY ENCLOSED BY '"'
    IGNORE 1 LINES
;
/* Use a select to transform the data as it is more readable than
   load data substitutions */
INSERT INTO \`Order\` (
    SELECT id,
            STR_TO_DATE(TRIM('"' FROM datePlaced), '%Y-%m-%d %T'),
            STR_TO_DATE(NULLIF(TRIM('"' FROM dateShipped), 'Cancelled'),
                '%Y-%m-%d %T'),
            customer_id
        FROM OrderStaging
);

DROP TABLE OrderStaging;

/* Create a table for the raw data with types suited for them */
CREATE TABLE ProductStaging (
    id BIGINT UNSIGNED,
    name VARCHAR(100),
    price VARCHAR(30),
    quantity INTEGER
);
LOAD DATA LOCAL INFILE '/home/${USERNAME}/products.csv'
    INTO TABLE ProductStaging
    FIELDS TERMINATED BY ','
        OPTIONALLY ENCLOSED BY '"'
    IGNORE 1 LINES
```

```
;
/* Use a select to transform the data as it is more readable
   than load data substitutions */
INSERT INTO Product (
    SELECT id,
            TRIM('"' FROM name),
            /* replace '$', ',', and '"' with empty string */
            CAST(REGEXP_REPLACE(price, '[$,"]', '') AS DECIMAL(6,2)),
            quantity
        FROM ProductStaging
);
INSERT INTO PriceHistory (oldPrice, newPrice, ts, product_id) (
    SELECT NULL,
            /* replace '$', ',', and '"' with empty string */
            CAST(REGEXP_REPLACE(price, '[$,"]', '') AS DECIMAL(6,2)),
            /* No data given so this is the 'new' official price as of NOW */
            NOW(),
            id
        FROM ProductStaging
);

DROP TABLE ProductStaging;

/* Create a table for the raw data with types suited for them */
CREATE TABLE OrderlineStaging (
    order_id BIGINT UNSIGNED,
    product_id BIGINT UNSIGNED
);
LOAD DATA LOCAL INFILE '/home/${USERNAME}/orderlines.csv'
    INTO TABLE OrderlineStaging FIELDS TERMINATED BY ','
        OPTIONALLY ENCLOSED BY '"'
    IGNORE 1 LINES
;
/* Use a select to transform the data as it is more readable
   than load data substitutions */
INSERT INTO Orderline (
    SELECT order_id,
            product_id,
            COUNT(*)
        FROM OrderlineStaging
        GROUP BY order_id, product_id
);

DROP TABLE OrderlineStaging;
SUBEOF

sudo -u ${USERNAME} cat > /home/${USERNAME}/views.sql <<SUBEOF
SOURCE /home/${USERNAME}/etl.sql

USE POS;

-- Handy view that will be used once ever
CREATE OR REPLACE VIEW v_ProductBuyers
```

```
    AS SELECT P.id productID, P.name productName,
            GROUP_CONCAT(
                DISTINCT CONCAT(C.id, ' ', C.firstName, ' ', C.lastName)
                ORDER BY C.id
                SEPARATOR ','
            ) customers
        FROM Customer C
        JOIN \`Order\` O ON O.customer_id = C.id
        JOIN Orderline OL ON OL.order_id = O.id
        -- Could benchmark this against an initial left join
        RIGHT JOIN Product P ON P.id = OL.product_id
        GROUP BY P.id, P.name
        ORDER BY P.id
;

-- Here it is, the single use of the view to create an MV table
CREATE TABLE IF NOT EXISTS mv_ProductBuyers ENGINE=InnoDB
    AS SELECT * FROM v_ProductBuyers
;

-- Boost performance by indexing the id
CREATE INDEX IF NOT EXISTS i_mv_ProductBuyers__productID
    ON mv_ProductBuyers (productID)
;

-- Prepare for code blocks
DELIMITER ||

-- Create procedure for shared update logic
CREATE OR REPLACE PROCEDURE p_update_customer_list(
    IN target_product_id BIGINT
)
BEGIN
    -- Refresh the list of customers that have ordered product
    UPDATE mv_ProductBuyers
        SET customers = (
            SELECT GROUP_CONCAT(
                DISTINCT CONCAT(C.id, ' ',
                    C.firstName, ' ', C.lastName
                )
                ORDER BY C.id
                SEPARATOR ','
            )
            FROM \`Order\` O
            JOIN (
                SELECT DISTINCT OL.order_id id FROM Orderline OL
                    WHERE OL.product_id = target_product_id
            ) PO ON PO.id = O.id
            -- At this point we have all product's orders
            JOIN Customer C ON C.id = O.customer_id
        )
        WHERE productID = target_product_id
    ;
```

```
END ||

-- Add triggers for both insert and delete to update mv_ProductBuyers
-- using p_update_customer_list
CREATE OR REPLACE TRIGGER t_mv_ProductBuyers__Orderline_insert
    AFTER INSERT ON Orderline
        FOR EACH ROW
        BEGIN
            CALL p_update_customer_list(NEW.product_id);
        END;
        ||

CREATE OR REPLACE TRIGGER t_mv_ProductBuyers__Orderline_delete
    AFTER DELETE ON Orderline
        FOR EACH ROW
        BEGIN
            CALL p_update_customer_list(OLD.product_id);
        END;
        ||

-- Only update price history when product price changes
CREATE OR REPLACE TRIGGER t_PriceHistory__Product_update
    AFTER UPDATE ON Product
        FOR EACH ROW
        BEGIN
            IF OLD.currentPrice != NEW.currentPrice THEN
                INSERT INTO PriceHistory(oldPrice, newPrice, ts, product_id)
                    VALUES(OLD.currentPrice, NEW.currentPrice, NOW(), OLD.id)
                ;
            END IF;
        END;
        ||

-- Done with code blocks
DELIMITER ;

-- Verify creation success
SHOW CREATE VIEW v_ProductBuyers;
SHOW CREATE TABLE mv_ProductBuyers;
SELECT * FROM mv_ProductBuyers LIMIT 5;

-- Prove price history updates only when it is supposed to
SELECT * FROM PriceHistory WHERE product_id = 3;
UPDATE Product SET currentPrice = currentPrice + 10.00 WHERE id = 3;
SELECT * FROM PriceHistory WHERE product_id = 3;
UPDATE Product SET availableQuantity = 0 WHERE id = 3;
SELECT * FROM PriceHistory WHERE product_id = 3;

-- Prove the MV updates eagerly when order lines are added/removed
SELECT * FROM mv_ProductBuyers WHERE productID = 3;
INSERT INTO Orderline VALUES (1,3,1);
SELECT * FROM mv_ProductBuyers WHERE productID = 3;
DELETE FROM Orderline WHERE order_id = 1 AND product_id = 3;
```

```
SELECT * FROM mv_ProductBuyers WHERE productID = 3;

SUBEOF

chown ${USERNAME}:${USERNAME} /home/${USERNAME}/etl.sql
chmod 755 /home/${USERNAME}/etl.sql
chown ${USERNAME}:${USERNAME} /home/${USERNAME}/views.sql
chmod 755 /home/${USERNAME}/views.sql

# TODO create script files for galera_config, galera_bootstrap, galera_join
# instead of running that stuff automatically overly manually
cat > /root/galera_config.sh <<SUBEOF
#!/bin/bash
# shebang so the shell knows this is a shell script
# \$1=THISPRIVATEIP \$2=NODEAPRIVATEIP \$3=NODEBPRIVATEIP \$4=NODECPRIVATEIP
# \$5=A|B|C

CLNAME='\"FurnitureCluster\"'
CLADDR="\$2,\$3,\$4"

DSE="default_storage_engine"
WCN="wsrep_cluster_name"
WCA="wsrep_cluster_address"
WNA="wsrep_node_address"
WNN="wsrep_node_name"
IALM="innodb_autoinc_lock_mode"

# Change configuration file to set host address and peer addresses
# ensure replication is enabled
sed -i "s/^#*\(wsrep_on[[:space:]]*= \)\([A-Z]*\)/\1ON/" \
/etc/mysql/mariadb.conf.d/60-galera.cnf
# uncomment and add bind address
sed -i "s/^#*\(bind-address[[:space:]]*= \)\([0-9\.]*\)/\1\${1}/" \
/etc/mysql/mariadb.conf.d/60-galera.cnf
# uncomment and ensure lock mode
sed -i "s/^#*\(\$IALM[[:space:]]*= \)\([0-9\.]*\)/\12/" \
/etc/mysql/mariadb.conf.d/60-galera.cnf
# uncomment and ensure log format is row
sed -i "s/^#*\(binlog_format[[:space:]]*= \)\([a-zA-Z]*\)/\1row/" \
/etc/mysql/mariadb.conf.d/60-galera.cnf
# uncomment and ensure engine is InnoDB
sed -i "s/^#*\(\$DSE[[:space:]]*= \)\([a-zA-Z]*\)/\1InnoDB/" \
/etc/mysql/mariadb.conf.d/60-galera.cnf
# uncomment and add cluster name
sed -i "s/^#*\(\$WCN[[:space:]]*= \)\(\"[a-zA-Z\s]*\"\)/\1\${CLNAME}/" \
/etc/mysql/mariadb.conf.d/60-galera.cnf
# uncomment and add cluster addresses
sed -i "s/^#*\(\$WCA[[:space:]]*= gcomm:\/\/\)\([0-9\.,]*\)/\1\${CLADDR}/" \
/etc/mysql/mariadb.conf.d/60-galera.cnf

# Add additional options
# Add node address
sed -i "/\[galera\].*/a\
```

```
\$WNA       = \$1" /etc/mysql/mariadb.conf.d/60-galera.cnf
# Add node name
sed -i "/\[galera\].*/a\
\$WNN          = Node\$5"  /etc/mysql/mariadb.conf.d/60-galera.cnf
# Add replication library path
sed -i "/\[galera\].*/a\
wsrep_provider           = \"\/usr\/lib\/galera\/libgalera_smm.so\"" \
 /etc/mysql/mariadb.conf.d/60-galera.cnf

touch /root/6-galera-config
SUBEOF
chmod 700 /root/galera_config.sh

cat > /root/galera_bootstrap.sh <<SUBEOF
#!/bin/bash
# shebang so the shell knows this is a shell script
sudo galera_new_cluster
touch /root/7-galera-bootstrap
SUBEOF
chmod 700 /root/galera_bootstrap.sh

cat > /root/galera_join.sh <<SUBEOF
#!/bin/bash
# shebang so the shell knows this is a shell script
sudo systemctl start mariadb
touch /root/8-galera-join
SUBEOF
chmod 700 /root/galera_join.sh

cat > /root/run_etl.sh <<SUBEOF
#!/bin/bash
# shebang so the shell knows this is a shell script

sudo -u ${USERNAME} wget http://622.gomillion.org/data/126009757.zip -O \
    /home/${USERNAME}/126009757.zip
sudo -u ${USERNAME} unzip /home/${USERNAME}/126009757.zip -d /home/${USERNAME}

mariadb <<SUBSUBEOF
CREATE DATABASE POS;
CREATE USER '${USERNAME}'@'localhost' IDENTIFIED BY '${PASSWORD}';
GRANT ALL PRIVILEGES ON POS.* TO '${USERNAME}'@'localhost';
FLUSH PRIVILEGES;

SUBSUBEOF

mariadb -e "SET GLOBAL log_bin_trust_function_creators = 1;"
mariadb -e "SET GLOBAL local_infile = 1;"
sudo -u ${USERNAME} cat /home/${USERNAME}/views.sql \
    | sudo -u ${USERNAME} mariadb -u ${USERNAME} --password=${PASSWORD}
mariadb -e "SET GLOBAL log_bin_trust_function_creators = 0;"
mariadb -e "SET GLOBAL local_infile = 0;"

touch /root/9-etl-done
```

```
SUBEOF
chmod 700 /root/run_etl.sh

echo "SUCCESS"
EOF



IMAGEID="ami-0ec10929233384c7f"   # Ubuntu server 24.04 LTS
INSTTYPE="t3.micro"
KEYNAME="vockey"
SGNAME="GaleraSG"
SGDESC="A security group for allowing ssh and other galera cluster members"
SGIDPARSE="SecurityGroups[*].GroupId"
# This is only used for ssh. To make extra secure use your own IP address
MYIP="0.0.0.0/0" # use curl https://checkip.amazonaws.com if running from home
MYACCT="$(aws sts get-caller-identity --query "Account" --output text)"
PORTS=("tcp 3306" "tcp 4567" "udp 4567" "tcp 4568" "tcp 4444")

# Check for Galera security group and create if it doesn't exist
SGID=$(aws ec2 describe-security-groups \
    --filters Name=group-name,Values="$SGNAME" \
    --query "$SGIDPARSE" \
    --output text)
if [ $? -eq 0 ] && [ -n $SGID ];
then
    echo "Security Group '$SGNAME' Exists"
elif [ $? -eq 0 ] && [ -z $SGID ];
then
    echo "Creating Security Group '$SGNAME'"
    SGID=$(aws ec2 create-security-group \
        --group-name $SGNAME \
        --description "$SGDESC" \
        --query "GroupId" \
        --output text)
    aws ec2 authorize-security-group-ingress \
        --group-id "$SGID" \
        --protocol tcp \
        --port 22 \
        --cidr "$MYIP"
    # Add MariaDB and Galera ports
    for i in "${PORTS[@]}";
    do
        PAIR=( $i )
        aws ec2 authorize-security-group-ingress \
            --group-id "$SGID" \
            --ip-permissions \
                "$(ip_permissions_str $SGID ${PAIR[0]} ${PAIR[1]} $MYACCT)"
    done
fi

INSTANCEA="$( aws ec2 run-instances \
    --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=A}]" \
```

```
    --image-id "$IMAGEID" \
    --count 1 \
    --instance-type "$INSTTYPE" \
    --key-name "$KEYNAME" \
    --security-group-ids "$SGID" \
    --query "Instances[0].InstanceId" \
    --iam-instance-profile Name="LabInstanceProfile" \
    --user-data "${STARTUP}" \
    --output text )"
INSTANCEB="$( aws ec2 run-instances \
    --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=B}]" \
    --image-id "$IMAGEID" \
    --count 1 \
    --instance-type "$INSTTYPE" \
    --key-name "$KEYNAME" \
    --security-group-ids "$SGID" \
    --query "Instances[0].InstanceId" \
    --iam-instance-profile Name="LabInstanceProfile" \
    --user-data "${STARTUP}"  \
    --output text )"
INSTANCEC="$( aws ec2 run-instances \
    --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=C}]" \
    --image-id "$IMAGEID" \
    --count 1 \
    --instance-type "$INSTTYPE" \
    --key-name "$KEYNAME" \
    --security-group-ids "$SGID" \
    --query "Instances[0].InstanceId" \
    --iam-instance-profile Name="LabInstanceProfile" \
    --user-data "${STARTUP}"  \
    --output text )"

sleep 1

# Wait for instance A to boot
while [ $(aws ec2 describe-instances \
    --filters "Name=instance-id,Values=$INSTANCEA" \
    --query "Reservations[].Instances[].State.Name" \
    --output text ) != "running" ];
do
    echo "Wait for instance A to finish boot..."
    sleep 10
done
INSTANCEAIP=$(aws ec2 describe-instances \
    --filters "Name=instance-id,Values=$INSTANCEA" \
    --query "Reservations[].Instances[].PrivateIpAddress" \
    --output text | sed "s/-/\./g" )

echo "Instance A has started with local IP $INSTANCEAIP"

# Wait for instance B to boot
while [ $(aws ec2 describe-instances \
    --filters "Name=instance-id,Values=$INSTANCEB" \
```

```
    --query "Reservations[].Instances[].State.Name" \
    --output text ) != "running" ];
do
    echo "Wait for instance B to finish boot..."
    sleep 10
done
INSTANCEBIP=$(aws ec2 describe-instances \
    --filters "Name=instance-id,Values=$INSTANCEB" \
    --query "Reservations[].Instances[].PrivateIpAddress" \
    --output text | sed "s/-/\./g" )

echo "Instance B has started with local IP $INSTANCEBIP"

# Wait for instance C to boot
while [ $(aws ec2 describe-instances \
    --filters "Name=instance-id,Values=$INSTANCEC" \
    --query "Reservations[].Instances[].State.Name" \
    --output text ) != "running" ];
do
    echo "Wait for instance C to finish boot..."
    sleep 10
done
INSTANCECIP=$(aws ec2 describe-instances \
    --filters "Name=instance-id,Values=$INSTANCEC" \
    --query "Reservations[].Instances[].PrivateIpAddress" \
    --output text | sed "s/-/\./g" )

echo "Instance C has started with local IP $INSTANCECIP"

# Wait for instance A to finish setup
COMMID=$( aws ssm send-command \
    --instance-ids "$INSTANCEA" \
    --document-name "AWS-RunShellScript" \
    --parameters \
        commands="tail -f /var/log/user-data.log | grep -m 1 "SUCCESS"" \
    --region us-east-1 \
    --query "Command.CommandId" \
    --output text )

sleep 1

while [ $(aws ssm get-command-invocation \
    --command-id "$COMMID" \
    --instance-id "$INSTANCEA" \
    --query "Status" \
    --output text ) != "Success" ];
do
    echo "Wait for instance A to finish setup..."
    sleep 10
done
echo "A has finished setup"

# Run configuration for instance A
```

```
COMMID=$( aws ssm send-command \
    --instance-ids "$INSTANCEA" \
    --document-name "AWS-RunShellScript" \
    --parameters \
        commands="/root/galera_config.sh $INSTANCEAIP $INSTANCEAIP \
            $INSTANCEBIP $INSTANCECIP A" \
    --region us-east-1 \
    --query "Command.CommandId" \
    --output text )

sleep 1

# Wait for instance A to finish config
while [ $(aws ssm get-command-invocation \
    --command-id "$COMMID" \
    --instance-id "$INSTANCEA" \
    --query "Status" \
    --output text ) != "Success" ];
do
    echo "Wait for instance A to finish config..."
    sleep 10
done
echo "A has finished config"

# Run bootstrap for instance A
COMMID=$( aws ssm send-command \
    --instance-ids "$INSTANCEA" \
    --document-name "AWS-RunShellScript" \
    --parameters \
        commands="/root/galera_bootstrap.sh" \
    --region us-east-1 \
    --query "Command.CommandId" \
    --output text )

sleep 1

# Wait for instance A to finish bootstrap
while [ $(aws ssm get-command-invocation \
    --command-id "$COMMID" \
    --instance-id "$INSTANCEA" \
    --query "Status" \
    --output text ) != "Success" ];
do
    echo "Wait for instance A to finish bootstrap..."
    sleep 10
done
echo "Instance A has finished bootstrap"

# Run config for instance B
COMMID=$( aws ssm send-command \
    --instance-ids "$INSTANCEB" \
    --document-name "AWS-RunShellScript" \
    --parameters \
```

```
        commands="/root/galera_config.sh $INSTANCEBIP $INSTANCEAIP \
            $INSTANCEBIP $INSTANCECIP B" \
    --region us-east-1 \
    --query "Command.CommandId" \
    --output text )

sleep 1

# Wait for instance B to finish config
while [ $(aws ssm get-command-invocation \
    --command-id "$COMMID" \
    --instance-id "$INSTANCEB" \
    --query "Status" \
    --output text ) != "Success" ];
do
    echo "Wait for instance B to finish config..."
    sleep 10
done
echo "Instance B has finished config"

# Run join for instance B
COMMID=$( aws ssm send-command \
    --instance-ids "$INSTANCEB" \
    --document-name "AWS-RunShellScript" \
    --parameters \
        commands="/root/galera_join.sh" \
    --region us-east-1 \
    --query "Command.CommandId" \
    --output text )

sleep 1

# Wait for instance B to finish join
while [ $(aws ssm get-command-invocation \
    --command-id "$COMMID" \
    --instance-id "$INSTANCEB" \
    --query "Status" \
    --output text ) != "Success" ];
do
    echo "Wait for instance B to finish join..."
    sleep 10
done
echo "Instance B has finished join"

# Run config for instance C
COMMID=$( aws ssm send-command \
    --instance-ids "$INSTANCEC" \
    --document-name "AWS-RunShellScript" \
    --parameters \
        commands="/root/galera_config.sh $INSTANCECIP $INSTANCEAIP \
            $INSTANCEBIP $INSTANCECIP C" \
    --region us-east-1 \
    --query "Command.CommandId" \
```

```
    --output text )

sleep 1

# Wait for instance C to finish config
while [ $(aws ssm get-command-invocation \
    --command-id "$COMMID" \
    --instance-id "$INSTANCEC" \
    --query "Status" \
    --output text ) != "Success" ];
do
    echo "Wait for instance C to finish config..."
    sleep 10
done
echo "Instance C has finished config"

# Run join for instance C
COMMID=$( aws ssm send-command \
    --instance-ids "$INSTANCEC" \
    --document-name "AWS-RunShellScript" \
    --parameters \
        commands="/root/galera_join.sh" \
    --region us-east-1 \
    --query "Command.CommandId" \
    --output text )

sleep 1

# Wait for instance C to finish join
while [ $(aws ssm get-command-invocation \
    --command-id "$COMMID" \
    --instance-id "$INSTANCEC" \
    --query "Status" \
    --output text ) != "Success" ];
do
    echo "Wait for instance C to finish join..."
    sleep 10
done
echo "Instance C has finished join"

# Run etl on instance A
COMMID=$( aws ssm send-command \
    --instance-ids "$INSTANCEA" \
    --document-name "AWS-RunShellScript" \
    --parameters \
        commands="/root/run_etl.sh" \
    --region us-east-1 \
    --query "Command.CommandId" \
    --output text )

sleep 1

# Wait for instance A to finish etl
```

```
while [ $(aws ssm get-command-invocation \
    --command-id "$COMMID" \
    --instance-id "$INSTANCEA" \
    --query "Status" \
    --output text ) != "Success" ];
do
    echo "Wait for instance A to finish etl..."
    sleep 10
done
echo "Instance A has finished etl"

echo "SUCCESS"

# TESTS FOR RUNNING
# Node A:
# SHOW STATUS LIKE 'wsrep_cluster_size';
# DROP DATABASE POS;
# sudo -u dvansteenwyk cat /home/dvansteenwyk/views.sql | sudo -u dvansteenwyk mariadb -u
dvansteenwyk --password="notthepassword"
# Node B:
# SHOW TABLES IN POS;
# Node C:
# SELECT COUNT(*) FROM Orderline;
# Node A:
# INSERT INTO Customer(firstName, lastName, email, phone, address1, address2, zip) VALUES
("Peter", "McCrach", "pmccrach10001@gmail.com", NULL, "1234 Incremental Dr", NULL, 10001);
# Node B:
# SELECT * FROM Customer WHERE email = "pmccrach10001@gmail.com";
# UPDATE Customer SET firstName = "Pete" WHERE email = "pmccrach10001@gmail.com";
# Node C:
# SELECT * FROM Customer WHERE email = "pmccrach10001@gmail.com";
# DELETE FROM Customer WHERE email = "pmccrach10001@gmail.com";
# Node A:
# SELECT * FROM Customer WHERE email = "pmccrach10001@gmail.com";
```

4. Video Outline (with Timestamps):

- Time 0:00 - Intro & Cluster Architecture (Instances, Ports, and Self-Referencing rule).
- Time 0:50 - Cluster Health Verification (Command and result showing size 3).
- Time 1:20 - Restore: Rebuild from Node A (Dropping DB and running script).
- Time 3:15 - Restore: Structure & Data Sync (Verifying on Nodes B and C).
- Time 4:20 - Multi-Master: Write to Node A (Insert test).
- Time 5:30 - Multi-Master: Update on Node B (Verify insert & perform update).
- Time 6:20 - Multi-Master: Delete on Node C (Verify update & perform delete).
- Time 7:10 - AI Disclosure.
- Time 8:00 - Reflection Answer.

5. Should you have any issues playing the video, please message me on Discord, send me an email, or send me a message in Canvas. A message left in a comment on this assignment will not notify me. I have collected proof this time that the video has been uploaded correctly, set to unlisted, and can play without error using the given link, should proof be needed.

Donald Van Steenwyk

2026 03 23 00 15 36

Saved as private

Details — Video elements — Checks — Visibility

Visibility

Choose when to publish and who can see your video

Save or publish

Make your video **public**, **unlisted**, or **private**

- Private
  Only you and people you choose can watch your video
- Unlisted
  Anyone with the video link can watch your video
- Public
  Everyone can watch your video

Set as instant Premiere

Uploading video...

2026 03 23 00 15 36

Video link

https://youtu.be/6q3lAR-Ldjg

Schedule

Select a date to make your video **public**.

Before you publish, check the following:

Do kids appear in this video?

Make sure you follow our policies to protect minors from harm, exploitation, bullying, and violations of labor law. Learn more

Looking for overall content guidance?

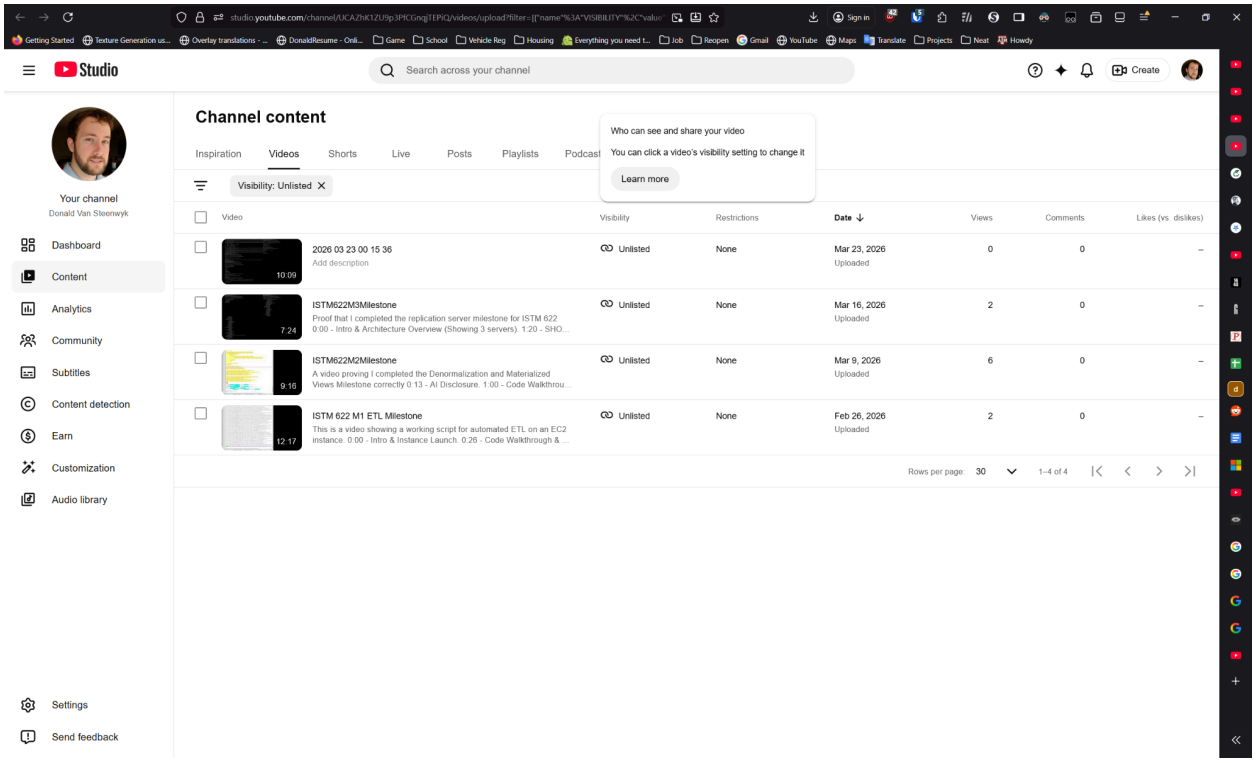
Our Community Guidelines can help you avoid trouble and ensure that YouTube remains a safe and vibrant community. Learn more

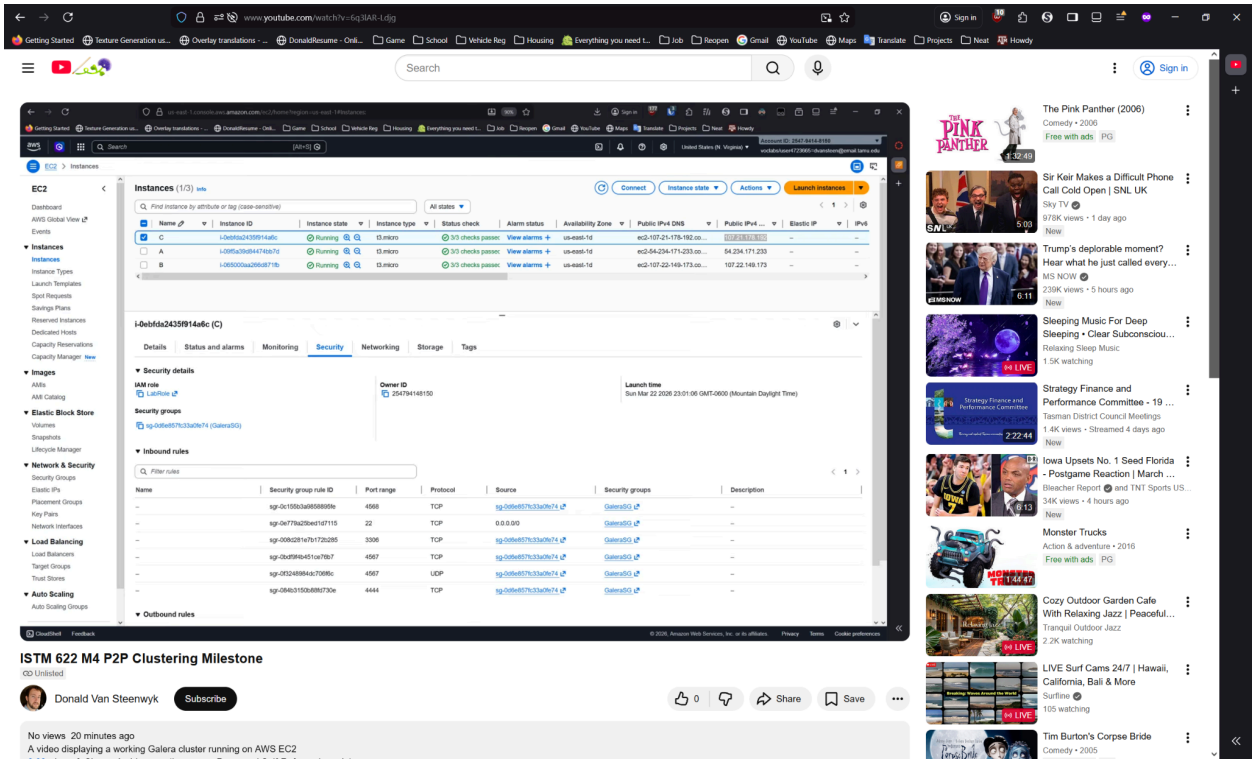
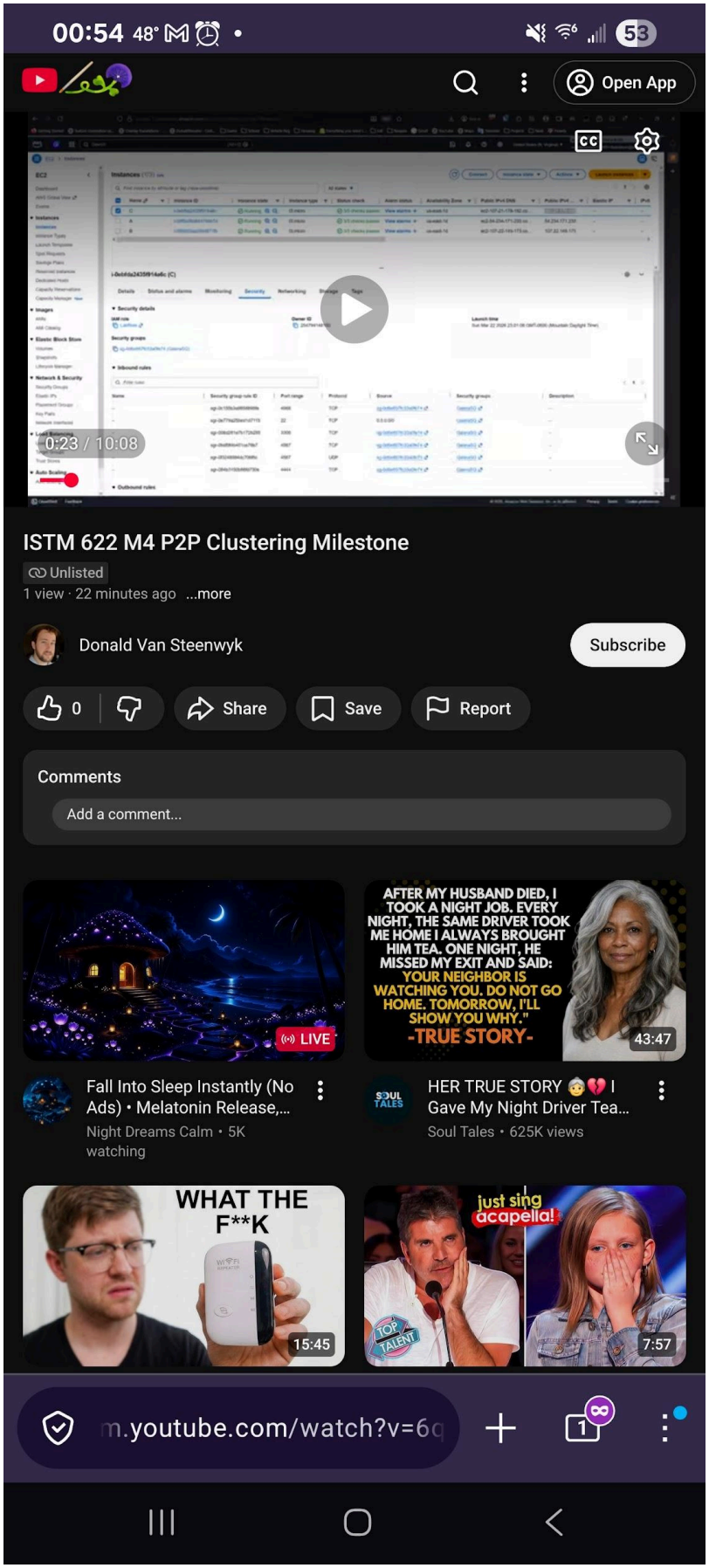
Uploading 41% ... 2 minutes left

Back

Save

Donald Van Steenwyk





Donald Van Steenwyk